LifeSynch AI — Source Code & Architecture Reference

Detailed File, Module, Database, and API Technical Catalog

Project: LifeSynch AI — Autonomous Academic & Personal Responsibility Hub
Repository: `minproj` (Monorepo with `frontend`, `backend`, `shared`, `extension`)

Table of Contents

1. Monorepo Directory Structure
2. Shared Workspace (`shared/`)
3. Backend Architecture (`backend/src/`)
   - 3.1 Entry Points & App Configuration
   - 3.2 Routes & API Endpoints
   - 3.3 Controllers
   - 3.4 Middlewares
   - 3.5 Services Layer
   - 3.6 Database SQL Schema & Supabase Client
   - 3.7 Test & Verification Scripts
4. Frontend Architecture (`frontend/src/`)
   - 4.1 Entry Points & Routing
   - 4.2 Authentication & State Context
   - 4.3 Components & UI Modules
   - 4.4 Frontend Services & Utilities
5. Chrome Extension (`extension/`)
6. Complete Database Schema Reference
7. Environment Variables Reference

---

1. Monorepo Directory Structure

The project is structured as an npm workspaces monorepo:

```
minproj/
├── backend/                # Express REST API, background sync, LLM pipelines
│   ├── src/
│   │   ├── config/         # Environment and Supabase configurations
│   │   ├── controllers/    # Route request/response handlers
│   │   ├── db/             # PostgreSQL DDL schema definition
│   │   ├── middlewares/    # Auth, CORS, validation, and error middlewares
│   │   ├── routes/         # Express router endpoints
│   │   ├── scripts/        # Standalone verification and test suites
│   │   ├── services/       # Business logic (AI, DB, Google, Sync, Reminders)
│   │   └── utils/          # JWT helpers and structured logger
│   ├── package.json
│   └── tsconfig.json
├── frontend/               # React 18 + Vite + Tailwind CSS Single-Page Application
│   ├── src/
│   │   ├── components/     # Reusable UI views (Auth, Dashboard, Assistant, Calendar)
│   │   ├── context/        # Global React AuthContext and session state
│   │   ├── services/       # API HTTP client, caching, intelligent search
│   │   └── utils/          # Text formatters and "Worth Knowing" helpers
│   ├── index.html
│   ├── package.json
│   ├── tailwind.config.js
│   └── vite.config.ts
├── shared/                 # Shared TypeScript models, Zod schemas, and utilities
│   ├── src/
│   │   ├── schemas/        # Zod runtime validation schemas
│   │   ├── services/       # Shared category and discovery rules
│   │   └── types/          # TypeScript interfaces (Task, Email, Event, User, API)
│   ├── package.json
│   └── tsconfig.json
├── extension/              # Manifest V3 browser extension for quick overlay
│   ├── manifest.json
│   ├── popup.html
│   └── popup.js
└── docs/                   # Documentation, reports, and architectural diagrams
    └── final-project-documentation/
```

2. Shared Workspace (`shared/`)

The `shared/` package ensures single-source-of-truth type safety across backend and frontend.

File Path	Description & Core Exports
`shared/src/types/task.ts`	Exports `Responsibility`, `TaskPriority` (`'high' | 'medium' | 'low'`), `TaskCategory` (`'academic' | 'work' | 'personal' | 'financial'`), `TaskStatus` (`'pending' | 'in_progress' | 'completed' | 'cancelled'`).
`shared/src/types/auth.ts`	Exports `User`, `ConnectedAccount`, `AuthSession`, `OAuthTokenPayload`.
`shared/src/types/email.ts`	Exports `IngestedEmail`, `EmailHeader`, `EmailClassification`.
`shared/src/types/event.ts`	Exports `IngestedCalendarEvent`, `EventTime`, `EventStatus`.
`shared/src/types/reminder.ts`	Exports `ReminderItem`, `NotificationPayload`, `ReminderBucket` (`'overdue' | 'due_today' | 'upcoming'`).
`shared/src/types/assistant.ts`	Exports `AssistantQueryRequest`, `AssistantQueryResponse`, `AssistantIntent`.
`shared/src/schemas/task.ts`	Exports Zod schemas (`responsibilitySchema`, `createTaskSchema`, `updateTaskSchema`) for runtime validation.
`shared/src/services/life-category.ts`	Categorization rules and keyword dictionaries used by both client search and backend classifiers.

3. Backend Architecture (`backend/src/`)

3.1 Entry Points & App Configuration

File	Purpose	Key Details
`backend/src/server.ts`	Server bootstrap entry point.	Initializes Express app, connects Supabase client, starts `AutoSyncService` 30-second recurring timer, and listens on port 5000 (or `PORT` env).
`backend/src/app.ts`	Express application factory.	Configures JSON body parser, CORS policies, request logging middleware, mounts `/api` routes, and attaches global error handling middleware.
`backend/src/config/env.ts`	Environment variable validation.	Reads `.env` and exports validated constants: `PORT` , `SUPABASE_URL` , `SUPABASE_SERVICE_ROLE_KEY` , `GOOGLE_CLIENT_ID` , `GOOGLE_CLIENT_SECRET` , `GOOGLE_REDIRECT_URI` , `LLM_API_KEY` .
`backend/src/config/supabase.ts`	Supabase client singleton.	Creates authenticated `@supabase/supabase-js` client instance using service role credentials for administrative and RLS-scoped database operations.

3.2 Routes & API Endpoints

The Express router (`backend/src/routes/index.ts`) aggregates and mounts all sub-routers under the `/api` prefix:

Route Path	Sub-Router File	HTTP Verb & Endpoint	Controller Function	Description
`/api/auth`	`auth.routes.ts`	`GET /google`	`AuthController.getGoogleAuthUrl`	Returns Google OAuth consent URL.
		`GET /google/callback`	`AuthController.handleGoogleCallback`	Exchanges OAuth code for tokens, creates/updates user, returns JWT session.
		`GET /me`	`AuthController.getCurrentUser`	Returns authenticated user profile and account connection status.
		`POST /logout`	`AuthController.logout`	Terminates session and invalidates cookie.
`/api/sync`	`sync.routes.ts`	`POST /trigger`	`SyncController.triggerSync`	Manually triggers immediate sync for current user.
		`GET /status`	`SyncController.getSyncStatus`	Returns current sync state (`idle` , `syncing` , `error`) and last synced timestamps.
`/api/responsibilities`	`task.routes.ts`	`GET /`	`TaskController.listResponsibilities`	Lists responsibilities with filtering by category, status, and priority.
		`GET /:id`	`TaskController.getResponsibilityById`	Returns a single responsibility by UUID.
		`POST /`	`TaskController.createResponsibility`	Manually creates a new responsibility.
		`PATCH /:id`	`TaskController.updateResponsibility`	Updates title, status, priority, or deadline.
		`DELETE /:id`	`TaskController.deleteResponsibility`	Soft-deletes or removes a responsibility.
`/api/reminders`	`reminder.routes.ts`	`GET /`	`ReminderController.getActiveReminders`	Returns categorized reminders

Route Path	Sub-Router File	HTTP Verb & Endpoint	Controller Function	Description
				(overdue, due today, upcoming).
		`POST /dismiss/:id`	`ReminderController.dismissReminder`	Dismisses a specific notification item.
`/api/assistant`	`assistant.routes.ts`	`POST /query`	`AssistantController.handleQuery`	Natural language interface with live LLM / grounded fallback.
`/api/health`	`health.routes.ts`	`GET /`	`HealthController.checkHealth`	Returns system health, timestamp, and database connectivity.

3.3 Controllers

Controller File	Methods	Primary Role
`controllers/auth.controller.ts`	`getGoogleAuthUrl`, `handleGoogleCallback`, `getCurrentUser`, `logout`	Manages OAuth redirects, code exchange, JWT cookie issuance, and user session retrieval.
`controllers/sync.controller.ts`	`triggerSync`, `getSyncStatus`	Triggers manual background synchronization and returns state from `sync_state` table.
`controllers/task.controller.ts`	`listResponsibilities`, `getResponsibilityById`, `createResponsibility`, `updateResponsibility`, `deleteResponsibility`	Handles CRUD operations for responsibilities, enforces user ID filtering, and handles pagination.
`controllers/reminder.controller.ts`	`getActiveReminders`, `dismissReminder`	Calls `ReminderService` to resolve active notifications based on current IST time.
`controllers/assistant.controller.ts`	`handleQuery`	Parses user prompt, queries user database records, calls Gemini LLM or falls back to grounded parser, and returns formatted response.
`controllers/health.controller.ts`	`checkHealth`	Verifies server uptime and Supabase connectivity.

3.4 Middlewares

Middleware File	Exported Function	Functionality
`middlewares/auth.middleware.ts`	`requireAuth`	Verifies JWT from `Authorization: Bearer <token>` header or session cookie; extracts `userId` and attaches `req.user`. Returns HTTP 401 if unauthorized.
`middlewares/validate.middleware.ts`	`validateBody(schema)`	Intercepts incoming requests and validates `req.body` against a Zod schema. Returns HTTP 400 with field-specific errors if validation fails.
`middlewares/error.middleware.ts`	`errorHandler`	Global error boundary catching unhandled rejections, logging error stack, and returning sanitized HTTP 500 JSON responses.

3.5 Services Layer

The services layer contains the core business and integration logic:

AI Services (`backend/src/services/ai/`)

- `extraction.service.ts` : Interfaces with Google Gemini Flash (`@google/genai`). Builds prompts containing raw email text, applies structured JSON constraints, and validates output against Zod schema.
- `responsibility-generation.service.ts` : Orchestrates ingestion to extraction pipeline. Reads unclassified emails/events from DB, calls `extraction.service` , and writes structured items into `responsibilities` .
- `responsibility-matcher.ts` : Compares new candidate responsibilities with existing records using Levenshtein distance and fuzzy matching to prevent duplicates.
- `assistant.service.ts` : Powers "Ask LifeSynch AI". Implements dual-engine architecture:
  - Generative path: Uses Gemini with grounded context.
  - Deterministic fallback: Pattern-matches intent (`today` , `this_week` , `urgent` , `academic`) and generates grounded tabular answers directly from database records when LLM quota is exhausted.

Database Services (`backend/src/services/db/`)

- `user.service.ts` : Manages `users` table operations (create, find by email, update profile).
- `account.service.ts` : Manages `connected_accounts` table (saves tokens, checks expiry, updates refresh tokens).
- `email.service.ts` : Handles batch upsert and retrieval of raw messages in `emails` table.
- `calendar-event.service.ts` : Handles upsert and status tracking of events in `calendar_events` table.
- `sync-state.service.ts` : Updates and retrieves timestamps and status in `sync_state` table.

Google Services (`backend/src/services/google/`)

- `oauth.service.ts` : Generates Google OAuth authorization URL, exchanges authorization code for tokens, and manages token auto-refresh via Google OAuth2 client.
- `gmail.service.ts` : Calls Gmail REST API (`users.messages.list` , `users.messages.get`), traverses MIME multi-part trees, and decodes URL-safe base64 text.
- `calendar.service.ts` : Calls Google Calendar API (`events.list`), handles single event expansions, cancelled event flags, and all-day date normalizations.

Background & Notification Services (`backend/src/services/`)

- `auto-sync.service.ts` : Implements 30-second recurring timer (`setInterval`). Contains mutex lock (`isRunning`) to prevent overlapping cycles and uses `Promise.allSettled` for concurrent Gmail and Calendar synchronization.
- `reminder.service.ts` : Calculates deadline timestamps in Indian Standard Time (IST, UTC+05:30), categorizes active responsibilities into `due_today` , `upcoming` , and `overdue` , and dynamically excludes completed or cancelled tasks.
- `task.service.ts` : Manages responsibility status transitions (`pending` → `in_progress` → `completed` / `cancelled`).

3.6 Database SQL Schema & Supabase Client

- `backend/src/db/schema.sql` : Contains the complete PostgreSQL DDL defining the 7 tables, indexes, foreign key relationships with cascading deletes, triggers for `updated_at`, and Row Level Security (RLS) policies.
- `backend/src/config/supabase.ts` : Instantiates the Supabase client using `SUPABASE_URL` and `SUPABASE_SERVICE_ROLE_KEY`.

3.7 Test & Verification Scripts

The `backend/src/scripts/` directory contains automated test suites that verify system integrity without requiring manual UI clicking:

Script Name	npm Command	What It Verifies
`verify_security.ts`	`npm run test:security -w backend`	Verifies JWT verification, unauthorized access rejection, and user isolation.
`test_gmail_ingestion.ts`	`npm run test:gmail -w backend`	Verifies Gmail MIME decoding, pagination, and duplicate prevention.
`test_calendar_ingestion.ts`	`npm run test:calendar -w backend`	Verifies Calendar event ingestion, all-day IST handling, and cancelled events.
`test_auto_sync.ts`	`npm run test:auto-sync -w backend`	Verifies 30s timer execution, concurrency mutex lock, and error resilience.
`test_assistant.ts`	`npm run test:assistant -w backend`	Runs 21 test cases verifying conversational intent parsing and grounded fallback responses.
`test_reminders.ts`	`npm run test:reminders -w backend`	Verifies deadline calculation, IST offset handling, and exclusion of completed tasks.
`test_notification_display.ts`	`npx ts-node src/scripts/test_notification_display.ts`	Tests dynamic relative date handling for notifications without hardcoded dates.

---

4. Frontend Architecture (`frontend/src/`)

4.1 Entry Points & Routing

File	Purpose
`frontend/src/main.tsx`	React 18 DOM mount point. Wraps `<App />` inside React `<StrictMode>`.
`frontend/src/App.tsx`	Main application shell. Evaluates authentication state from `AuthContext` and conditionally renders `<LoginPage />` or `<AppLayout />`.
`frontend/src/components/layout/AppLayout.tsx`	Primary layout container containing `<Navbar />`, `<Sidebar />`, notification bell, sync status pill, and view routing (Dashboard, Calendar, Profile, Assistant).

4.2 Authentication & State Context

- `frontend/src/context/AuthContext.tsx` : React Context providing global user authentication state.
  - Exports `useAuth()` hook.
  - Manages `user`, `loading`, `isAuthenticated`, `loginWithGoogle()`, `logout()`.
  - Automatically verifies session cookie on mount via `GET /api/auth/me`.

4.3 Components & UI Modules

Component Directory	Component Name	Description
`components/auth/`	`LoginPage.tsx`	High-conversion login screen with Google OAuth button and feature highlights.
	`AuthCallback.tsx`	Handles Google OAuth redirect, extracts URL params, completes session handshake, and redirects to dashboard.
`components/dashboard/`	`PersonalIntelligenceHub.tsx`	Main dashboard view: quick stats cards, actionable responsibilities list, category filter pills, and Worth Knowing feed.
	`WorthKnowingDetailModal.tsx`	Modal displaying complete details and original email context for informational announcements.
`components/assistant/`	`AskAssistant.tsx`	Interactive sliding chat drawer for natural-language scheduling queries with quick suggestion chips.
`components/calendar/`	`CalendarEventModal.tsx`	Detailed view for calendar events showing start/end times, attendee status, and associated extracted tasks.
`components/common/`	`Navbar.tsx`	Header bar containing brand logo, global search bar, sync status badge, notification bell, and user avatar.
	`Sidebar.tsx`	Navigation drawer with links to Dashboard, Responsibilities, Calendar, Worth Knowing, and Settings.
	`Card.tsx`, `Button.tsx`, `Badge.tsx`	Reusable atomic UI components styled with Tailwind CSS.
`components/profile/`	`ProfilePage.tsx`	Displays user profile info, connected Google account email, and account disconnect button.
`components/search/`	`GlobalSearchResultsView.tsx`	Instant full-text search overlay filtering across responsibilities, emails, and calendar events.

4.4 Frontend Services & Utilities

Service / Utility	Role
`frontend/src/services/api.ts`	Axios / Fetch wrapper configured with base URL and credentials. Encapsulates all backend REST calls.
`frontend/src/services/dataCache.ts`	Client-side memory cache with TTL to prevent redundant network requests during navigation.
`frontend/src/services/globalSearchService.ts`	Fast client-side multi-field fuzzy search across active responsibilities.
`frontend/src/utils/textUtils.ts`	String helpers for capitalization, date formatting, and text truncation.
`frontend/src/utils/worthKnowingHelper.ts`	Helper functions to identify and format "Worth Knowing" informational notices.

5. Chrome Extension (`extension/`)

LifeSynch AI includes a companion Manifest V3 browser extension allowing users to view tasks and notifications without leaving their current tab:

- `manifest.json`: Configures Manifest V3 permissions (`storage`, `activeTab`) and background service worker.
- `popup.html` & `popup.js`: Mini-dashboard popup displaying upcoming deadlines, quick stats, and sync status directly from the backend API.

- `content.js` : Content script enabling quick-add overlays on supported web pages.

---

6. Complete Database Schema Reference

The Supabase PostgreSQL database contains **7 relational tables**:

```
+-------------------------------------------------------------------------+
|                    SUPABASE POSTGRESQL TABLES                           |
|                                                                         |
|  1. users              (id [PK, UUID], email, full_name, avatar_url)    |
|  2. connected_accounts (id [PK], user_id [FK], provider, access_token...) |
|  3. emails             (id [PK], user_id [FK], message_id, subject...)  |
|  4. calendar_events    (id [PK], user_id [FK], event_id, summary...)    |
|  5. responsibilities   (id [PK], user_id [FK], title, priority, due_date)|
|  6. notifications      (id [PK], user_id [FK], type, title, is_read...) |
|  7. sync_state         (id [PK], user_id [FK], sync_status, last_synced) |
+-------------------------------------------------------------------------+
```

Table Column Details:

1. `users`
   - `id` (UUID, Primary Key)
   - `email` (TEXT, Unique, Not Null)
   - `full_name` (TEXT)
   - `avatar_url` (TEXT)
   - `created_at` (TIMESTAMPTZ, Default: `now()`)
   - `updated_at` (TIMESTAMPTZ, Default: `now()`)

2. `connected_accounts`
   - `id` (UUID, Primary Key)
   - `user_id` (UUID, Foreign Key → `users.id` ON DELETE CASCADE)
   - `provider` (TEXT, Default: `'google'`)
   - `access_token` (TEXT, Not Null)
   - `refresh_token` (TEXT)
   - `token_expiry` (TIMESTAMPTZ)
   - `scopes` (TEXT[])
   - `created_at` (TIMESTAMPTZ)
   - `updated_at` (TIMESTAMPTZ)

3. `emails`
   - `id` (UUID, Primary Key)
   - `user_id` (UUID, Foreign Key → `users.id` ON DELETE CASCADE)
   - `message_id` (TEXT, Unique per user)
   - `thread_id` (TEXT)
   - `sender` (TEXT)
   - `recipient` (TEXT)
   - `subject` (TEXT)
   - `snippet` (TEXT)
   - `body_text` (TEXT)
   - `received_at` (TIMESTAMPTZ)
   - `is_processed` (BOOLEAN, Default: `false`)

4. `calendar_events`

- `id` (UUID, Primary Key)
- `user_id` (UUID, Foreign Key → `users.id` ON DELETE CASCADE)
- `event_id` (TEXT, Unique per user)
- `summary` (TEXT)
- `description` (TEXT)
- `location` (TEXT)
- `start_time` (TIMESTAMPTZ)
- `end_time` (TIMESTAMPTZ)
- `is_all_day` (BOOLEAN, Default: `false`)
- `status` (TEXT, Default: `'confirmed'`)

5. `responsibilities`

- `id` (UUID, Primary Key)
- `user_id` (UUID, Foreign Key → `users.id` ON DELETE CASCADE)
- `source_type` (TEXT: `'email'` | `'calendar'` | `'manual'`)
- `source_id` (TEXT)
- `title` (TEXT, Not Null)
- `description` (TEXT)
- `category` (TEXT: `'academic'` | `'work'` | `'personal'` | `'financial'`)
- `priority` (TEXT: `'high'` | `'medium'` | `'low'`)
- `status` (TEXT: `'pending'` | `'in_progress'` | `'completed'` | `'cancelled'`)
- `due_date` (TIMESTAMPTZ)
- `completed_at` (TIMESTAMPTZ)
- `metadata` (JSONB) — Contains `dueTime`, `confidenceScore`, `extractionRationale`, `itemType`
- `created_at` (TIMESTAMPTZ)
- `updated_at` (TIMESTAMPTZ)

6. `notifications`

- `id` (UUID, Primary Key)
- `user_id` (UUID, Foreign Key → `users.id` ON DELETE CASCADE)
- `responsibility_id` (UUID, Foreign Key → `responsibilities.id` ON DELETE SET NULL)
- `type` (TEXT: `'upcoming'` | `'due_today'` | `'overdue'`)
- `title` (TEXT, Not Null)
- `message` (TEXT)
- `is_read` (BOOLEAN, Default: `false`)
- `created_at` (TIMESTAMPTZ)

7. `sync_state`

- `id` (UUID, Primary Key)
- `user_id` (UUID, Foreign Key → `users.id` ON DELETE CASCADE)
- `provider` (TEXT, Default: `'google'`)
- `sync_status` (TEXT: `'idle'` | `'syncing'` | `'error'`)
- `last_email_sync` (TIMESTAMPTZ)
- `last_calendar_sync` (TIMESTAMPTZ)
- `error_message` (TEXT)
- `updated_at` (TIMESTAMPTZ)

---

7. Environment Variables Reference

Variable Name	Required By	Format / Example	Purpose
`PORT`	Backend	`5000`	HTTP port for the Express application server.
`NODE_ENV`	Backend	`development` / `production`	Execution mode.
`SUPABASE_URL`	Backend	`https://xyzcompany.supabase.co`	REST endpoint for Supabase PostgreSQL database.
`SUPABASE_SERVICE_ROLE_KEY`	Backend	`eyJhbGciOiJIUz...`	Elevated service role key for server-side database operations.
`GOOGLE_CLIENT_ID`	Backend	`123456...apps.googleusercontent.com`	Google Cloud OAuth 2.0 client credential.
`GOOGLE_CLIENT_SECRET`	Backend	`GOCSPX-...`	Google Cloud OAuth 2.0 client secret.
`GOOGLE_REDIRECT_URI`	Backend	`http://localhost:5000/api/auth/google/callback`	OAuth redirect endpoint registered in Google Cloud Console.
`LLM_API_KEY`	Backend	`AIzaSy...`	Canonical Google Gemini API key for LLM extraction and assistant.
`JWT_SECRET`	Backend	`supersecretjwtkey...`	Cryptographic secret for signing and verifying user session tokens.
`VITE_API_URL`	Frontend	`http://localhost:5000`	Base URL used by the React frontend to communicate with backend APIs.